



EE-129 Project 2 Report

Web Server with Login System

Yunpeng Wang

Tufts University

Department of Electrical and Computer Engineering

March 31, 2025

Contents

1	Introduction	2
2	Experimental Setup	3
2.1	System Configuration	3
2.2	Code Development Stages	4
2.2.1	Basic Server and Color Picker	5
2.2.2	Color Response Logic	6
2.2.3	Login System via POST	6
2.2.4	Multithreaded Client Handling	7
2.2.5	Command Line Interaction with select()	8
3	Results	9
3.1	Color Picker Homepage and Interaction	9
3.2	GET Request Behavior	10
3.3	Login Form and Credential Validation	11
3.4	Command Line Interaction	12
3.5	Key Observations	13
4	Discussion	14
5	Conclusion	15

1 Introduction

The goal of this project is to design and implement a basic web server from scratch using low-level Python socket programming. Unlike traditional web frameworks, this project requires a manual understanding of how HTTP requests are received, parsed, and responded to, simulating a real-world server's core functionality.

This assignment integrates several key concepts covered in EE-129, including the TCP/IP model, socket-level communication, HTTP protocol structure (GET and POST), and concurrent server design. It also emphasizes real-time user interaction, form submission processing, and dynamic HTML response generation.

Our server is capable of handling browser requests, serving an interactive color picker, and processing a login system with user credential validation. The login system tracks successful and failed attempts persistently using file I/O, and the server can respond to runtime commands like `help` and `exit` through stdin. Moreover, we extend the basic implementation by enabling multi-threaded concurrent client handling, making the server more scalable and responsive.

2 Experimental Setup

This section outlines the complete environment, tools, and implementation steps involved in developing our Python-based web server for EE-129 Project 2.

2.1 System Configuration

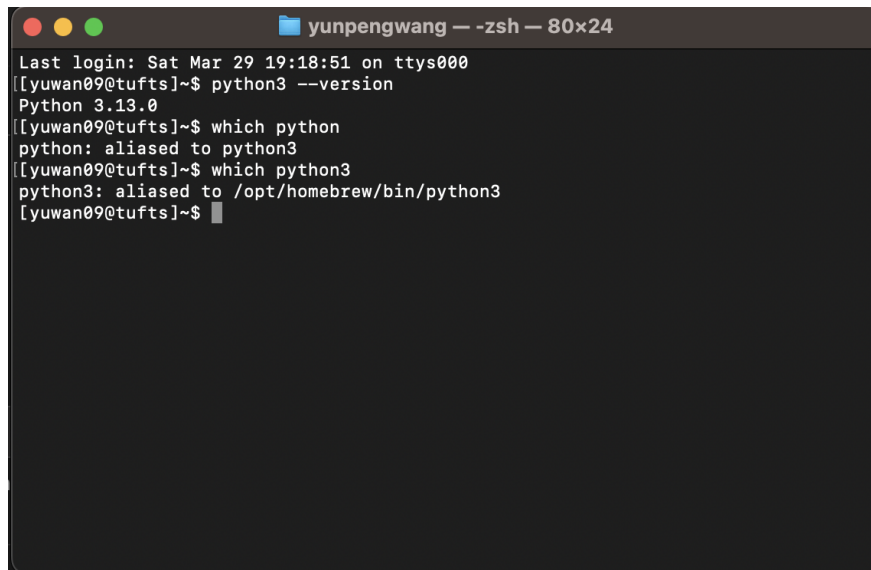
The server was developed and tested on a local macOS system using the Terminal and Visual Studio Code (VS Code) editor.

- **Operating System:** macOS 13.6
- **Python Version:** 3.13.0 (installed via Homebrew)
- **Editor:** Visual Studio Code with Python extension enabled

The Python version was verified in the terminal using the command:

```
python3 --version  
which python3
```

This confirmed that Python 3.13.0 from Homebrew was being used. (See Figure 1)

A screenshot of a macOS terminal window. The title bar shows 'yunpengwang -- zsh -- 80x24'. The terminal content shows the following commands and output:

```
Last login: Sat Mar 29 19:18:51 on ttys000  
[yuwan09@tufts]~$ python3 --version  
Python 3.13.0  
[yuwan09@tufts]~$ which python  
python: aliased to python3  
[yuwan09@tufts]~$ which python3  
python3: aliased to /opt/homebrew/bin/python3  
[yuwan09@tufts]~$
```

Figure 1: Verifying Python version and interpreter path

In Visual Studio Code, the Python interpreter was set to 3.13.0 (bottom right corner), and the final script was opened and executed from there. (See Figure 2)

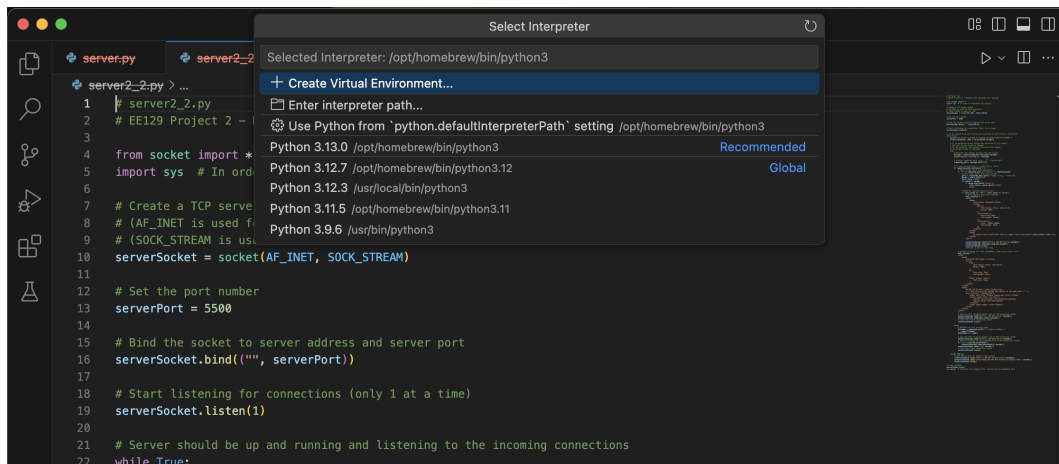


Figure 2: Visual Studio Code with selected Python interpreter and server code open

The server was executed in Terminal by running:

```
python3 server3_1.py
```

This started the server and listened on port 5500. (See Figure 3)



Figure 3: Server successfully listening at port 5500

2.2 Code Development Stages

The project was implemented incrementally across several scripts: `server.py`, `server2_2.py`, `server2_3.py`, and the final `server3_1.py`.

2.2.1 Basic Server and Color Picker

The initial server accepted a single connection and returned a color selection form when the browser requested the root URL “/”. The HTML form allowed the user to choose a color (red or green) and submit it using the GET method. (See Figure 4)

```
try:
    # Receives the request message from the client
    message = connectionSocket.recv(1024).decode()
    print("Request received:\n", message)

    # Extract requested path (like "/", "/red", etc.)
    requested_path = message.split()[1]

    # Check if path is root "/"
    if requested_path == "/":
        html_content = """
        <html>
            <head>
                <title>EE 129 Project 2</title>
                <style>
                    body {
                        font-family: Arial, sans-serif;
                        margin: 40px;
                    }
                    h1 {
                        font-size: 36px;
                        font-weight: bold;
                    }
                    label, select, input {
                        font-size: 16px;
                    }
                </style>
            </head>
            <body>
                <h1>EE 129 Project 2 Color Picker!</h1>
                <form action="/" method="get">
                    <label for="color">Please choose your color:</label>
                    <select name="color" id="color">
                        <option value="green" selected>Green</option>
                        <option value="red">Red</option>
                    </select>
                    <input type="submit" value="Submit">
                </form>
            </body>
        </html>
        """
```

Figure 4: Basic TCP server returning HTML form at root path

2.2.2 Color Response Logic

In the next stage, we handled GET parameters like `/?color=red` and dynamically returned the message `Your color is red!`, styling the color name appropriately using inline CSS. (See Figure 5)

```
# If user is accessing "/", show form or result
if requested_path.startswith("/"):
    # Check if URL has a color parameter
    if "?" in requested_path and "color=" in requested_path:
        # Parse color value from query string
        query = requested_path.split("?")[1] # e.g., "color=red"
        params = query.split("&")
        color_value = None
        for param in params:
            if param.startswith("color="):
                color_value = param.split("=")[1]
                break

        # Generate response based on color
        if color_value == "red" or color_value == "green":
            # Send response with color message
            html_response = f"""
            <html>
            <head>
```

Figure 5: Dynamic GET parameter handling for color feedback

2.2.3 Login System via POST

To support authentication, a new page at path `/login` was created. A POST form submitted username and password to the server, which validated credentials and dynamically updated success/failure statistics saved in `login_stats.txt`. (See Figure 6)

```

# Function to handle each client connection
def handle_client(connectionSocket):
    try:
        message = connectionSocket.recv(1024).decode()
        print("Request received:\n", message)

        # Handle POST login submission (form submitted from login page)
        if message.startswith("POST /login"):
            # Split full HTTP request into individual lines
            headers = message.split("\r\n")
            content_length = 0
            # Extract Content-Length to know how many bytes to expect in the body
            for h in headers:
                if h.lower().startswith("content-length:"):
                    content_length = int(h.split(":")[1].strip())
            # The HTTP body starts after a blank line (\r\n\r\n)
            body = message.split("\r\n\r\n")[1]
            # If body is not fully received (e.g., split into multiple packets), continue reading
            while len(body) < content_length:
                body += connectionSocket.recv(1024).decode()

            # Parse the form data: "username=user&password=pass" → {'username': ..., 'password': ...}
            params = dict(param.split("=") for param in body.split("&"))
            # Extract the submitted username and password from the parsed data
            username = params.get("username", "")
            password = params.get("password", "")

```

Figure 6: Handling POST request with login credentials and file-based statistics tracking

2.2.4 Multithreaded Client Handling

In the final server version, we introduced `threading.Thread` to handle each client connection in parallel, improving scalability and responsiveness. (See Figure 7)

```

for source in read_ready:
    # If the server socket is ready, it means a client is trying to connect
    if source == serverSocket:
        # Accept the incoming client connection
        connectionSocket, addr = serverSocket.accept()
        # Start a new thread to handle this client independently
        # Set daemon=True so the server can exit even if client threads are running
        client_thread = threading.Thread(target=handle_client, args=(connectionSocket,), daemon=True)
        client_thread.start()
    # If input is ready on stdin, read and handle the command
    elif source == sys.stdin:
        user_input = sys.stdin.readline().strip()
        # If user types "exit", shut down the server and terminate the program
        if user_input == "exit":
            print("Shutting down server...")
            serverSocket.close()
            sys.exit()
        # If user types "help", display available commands
        elif user_input == "help":
            print("Available commands:")
            print("  help - show this message")
            print("  exit - shut down the server")
        # Handle unknown commands
        else:
            print(f"{user_input}: Command Not Found")

```

Figure 7: Spawning new thread for each incoming client connection

2.2.5 Command Line Interaction with select()

The server also monitored stdin for user commands using `select.select()`, allowing real-time control over the server. The supported commands were `help` (prints usage) and `exit` (shuts down server). (See Figure 8)

```
while True:
    print("The server is recieving at http://127.0.0.1:5500/")
    # Use select to wait for input from either:
    # - the server socket (new client connection)
    # - standard input (commands like "exit", "help")
    read_ready, _, _ = select.select([inputs, []], [], [])
```

Figure 8: `select()` based parallel input monitoring for stdin and sockets

3 Results

This section presents the results from executing the implemented web server. All screenshots were taken during live testing using Google Chrome and Terminal on macOS. These results illustrate the dynamic server functionality and user interaction in response to both browser-based and command-line inputs.

3.1 Color Picker Homepage and Interaction

Upon navigating to `http://127.0.0.1:5500/`, the user is greeted with a color selection form and a login button. The page is rendered with clear layout and form elements. (See Figure 9)

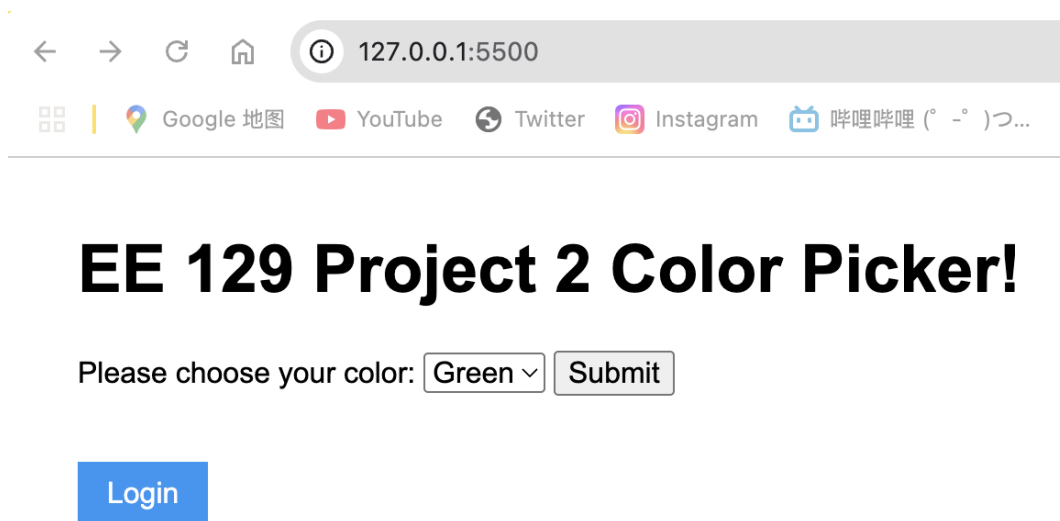


Figure 9: Homepage with color picker and login button

The color drop-down menu offers two options: red and green. The default selection is green, as shown in Figure 10.

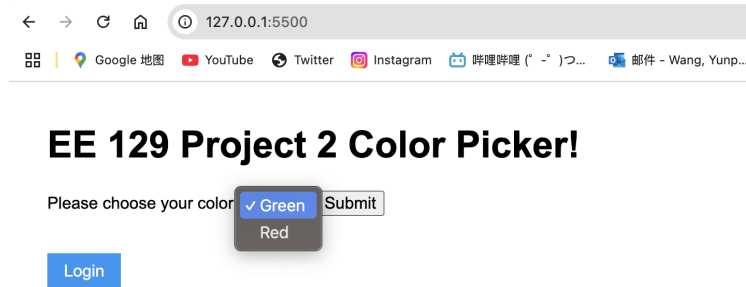


Figure 10: Drop-down menu showing color options: green and red

3.2 GET Request Behavior

When a color is selected and submitted, the server parses the GET request and returns a dynamic HTML page. As demonstrated in Figure 11, the phrase “Your color is red!” is returned, and the word **red** is colored accordingly.

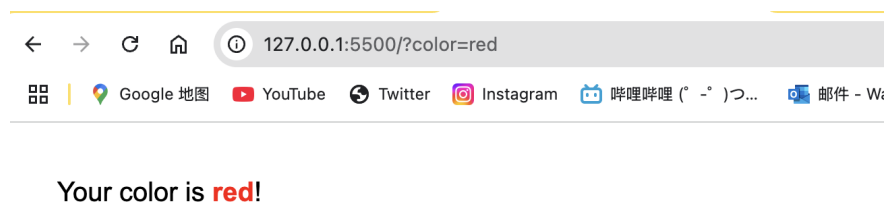


Figure 11: Dynamic HTML response for GET parameter `color=red`

Likewise, if green is selected, the response will dynamically highlight “green” in the sentence. (See Figure 12)

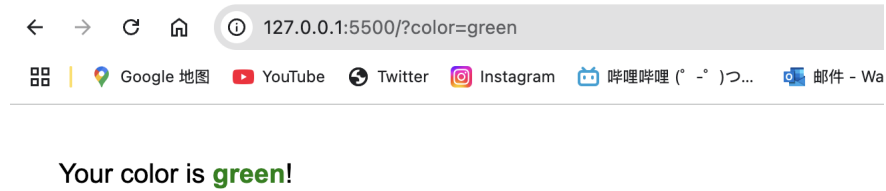


Figure 12: Dynamic HTML response for GET parameter `color=green`

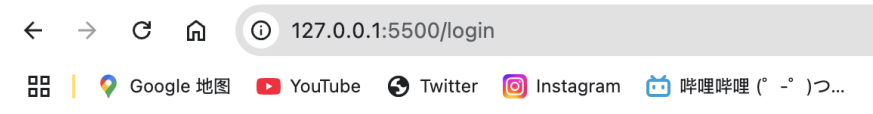
3.3 Login Form and Credential Validation

Clicking the login button leads to a dedicated login page where the user can enter credentials. The layout includes labeled input fields and a styled submit button. (See Figure 13)

A screenshot of a web browser window showing a login page. The address bar displays '127.0.0.1:5500/login?'. The page content includes a heading 'Login Page'. Below the heading, there are two text labels: 'Please enter your username:' followed by an empty text input field, and 'Please enter your password:' followed by an empty password input field. At the bottom of the form is a green rectangular button with the text 'Submit' in white.

Figure 13: Login form with input fields for username and password

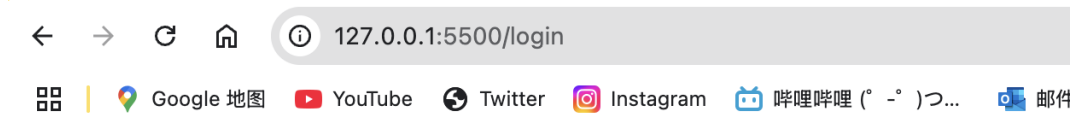
If incorrect credentials are submitted, the server returns a red-colored error message. (See Figure 14)



Login Failed

Figure 14: Login failed message rendered in red

If the correct credentials are submitted (`user` and `password`), the server responds with a success message and dynamically updated counters for login attempts. (See Figure 15)



Login Success

Successful logins: 4

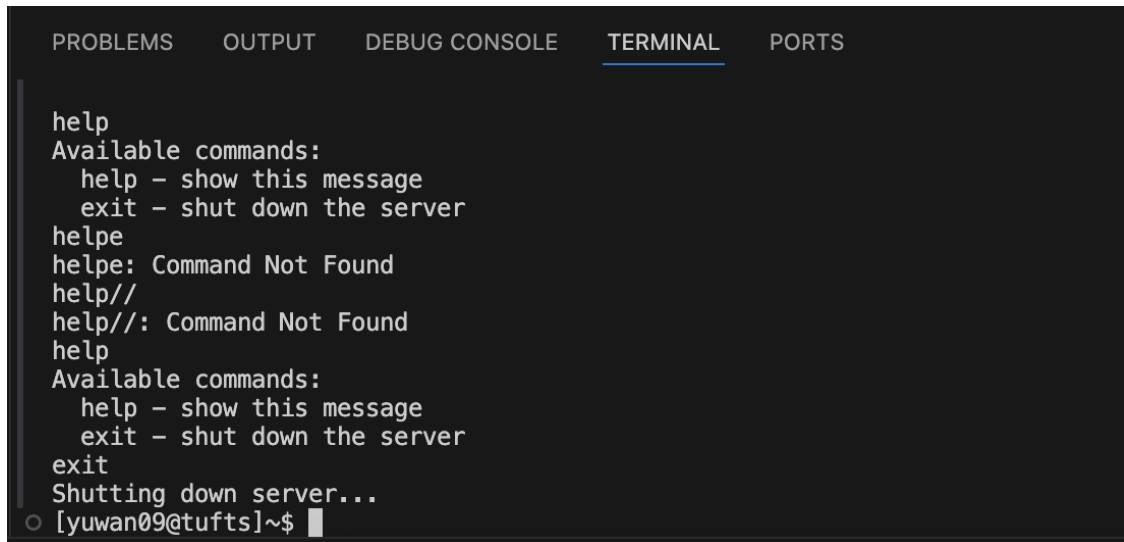
Failed logins: 3

Figure 15: Login success and persistent login statistics

3.4 Command Line Interaction

The server also accepts direct terminal input via standard input (stdin) while running. Figure 16 shows the command-line interaction:

- `help` prints available commands.
- `exit` gracefully terminates the server.
- Unknown commands return a formatted error message.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

help
Available commands:
  help - show this message
  exit - shut down the server
helpe
helpe: Command Not Found
help//
help//: Command Not Found
help
Available commands:
  help - show this message
  exit - shut down the server
exit
Shutting down server...
[yuwan09@tufts]~$
```

Figure 16: Command-line interface: help, exit, and unknown command handling

3.5 Key Observations

These results confirm that the server meets all functional requirements of Project 2:

- Dynamic GET and POST request handling
- HTML response rendering based on user input
- Persistent login statistics maintained using file I/O
- Responsive command-line control
- Clean handling of valid and invalid interactions

Together, these behaviors illustrate the practical implementation of a multi-threaded, socket-based web server supporting full HTTP request parsing and real-time user interaction.

4 Discussion

The results of this project provide strong evidence that the implemented server successfully fulfilled the core objectives of Project 2. The correct rendering of color picker interactions, the ability to respond to HTTP GET and POST requests, the persistent tracking of login attempts, and the inclusion of command-line control collectively demonstrate a robust understanding of low-level web server operations.

From a theoretical perspective, the project concretely connected several EE-129 concepts to practice. We gained firsthand experience in managing socket-level communication, parsing raw HTTP requests, building dynamic HTML responses, and handling multiple client connections using threading. The design of a multithreaded server that can concurrently accept client connections while simultaneously responding to user commands through stdin exemplifies advanced parallel processing in network applications.

One challenge encountered involved managing multiple client threads while allowing the server to accept terminal input concurrently. Initially, the program could not exit cleanly because background threads blocked the main loop. After testing, we realized the fix involved setting each spawned thread to `daemon=True`, ensuring they would not prevent program termination. Another unexpected behavior came from browser-prefetching requests (like repeated GET /favicon.ico) which had to be filtered or ignored in the server logic to prevent confusion.

Additionally, ensuring that login success and failure counters persisted across sessions required a careful design using file I/O. Handling this logic involved checking whether the stats file existed, parsing its content, and rewriting it consistently after each login attempt. This implementation emphasized the importance of robust file handling and data consistency.

From a usability standpoint, debugging visual design in HTML, such as properly styling the color response or aligning form components, required iterative refinement. We also learned how browser behaviors like auto-filling, pre-fetching, or modifying request headers could impact server testing.

This project solidified the students' ability to design, debug, and deploy a Python-based web server from scratch. The work provided not only an implementation challenge but a rich learning experience in integrating socket programming, system-level concurrency, HTTP semantics, and browser interactions.

5 Conclusion

This project challenged us to step beyond theoretical understanding and build a working web server entirely from scratch using Python’s socket library. Unlike using traditional web frameworks, we manually handled all HTTP request parsing, connection management, dynamic content rendering, and response generation.

Our server now supports a fully interactive experience. It includes a user-friendly color picker via GET requests, a complete login system using POST with credential verification, persistent tracking of login statistics using file I/O, and a command-line interface to control server behavior at runtime. The final enhancement—supporting multiple concurrent client connections using threads—makes the server scalable and responsive, suitable for handling real-world asynchronous traffic.

Throughout this project, we deepened our understanding of the TCP/IP protocol stack, concurrent programming with threads, and low-level web protocols. We also learned to anticipate and handle browser-side behaviors, asynchronous request timing, and error handling in network environments.

The project outcome was highly successful. We built a lightweight, extensible server that not only satisfies the project specification but also mimics key components of real-world server architectures. This project has enhanced our readiness for advanced topics in networking, backend systems, and distributed computing, and improved our confidence in working with foundational system-level programming tasks.

Appendix

References

- Python Socket Programming: <https://docs.python.org/3/library/socket.html>
- Python Threading Documentation: <https://docs.python.org/3/library/threading.html>
- W3Schools HTML Forms: https://www.w3schools.com/html/html_forms.asp *RFC2616—HTTP/1.1 Specification* : <https://www.rfc-editor.org/rfc/rfc2616>
- Python select Module: <https://docs.python.org/3/library/select.html>

Complete Final Code:

```
# EE129 Project 2 - Final Version with CLI Control + Multi-Client Support
```

```
from socket import *    # Import all socket module functions and constants (for TCP/IP ne
import sys              # Provides access to system-level functions like sys.exit()
import os               # Used to check file existence and interact with the file system
import threading        # For multi-threaded client handling
import select           # For monitoring both stdin and sockets
```

```
# Create a TCP server socket
# (AF_INET is used for IPv4 protocols)
# (SOCK_STREAM is used for TCP)
serverSocket = socket(AF_INET, SOCK_STREAM)
```

```
# Set the port number
serverPort = 5500
```

```
# Bind the socket to server address and server port
serverSocket.bind(("", serverPort))
```

```
# Start listening for connections (only 1 at a time, but we'll support multi-thread)
serverSocket.listen(5) # Allow up to 5 pending connections
```

```
# File path to store login stats
stats_file = "login_stats.txt"
```

```
# Initialize stats file if not exists
if not os.path.exists(stats_file):
    with open(stats_file, "w") as f:
```

```

        f.write("success=0\nfail=0\n")

# Function to handle each client connection
def handle_client(connectionSocket):
    try:
        message = connectionSocket.recv(1024).decode()
        print("Request received:\n", message)

        # Handle POST login submission (form submitted from login page)
        if message.startswith("POST /login"):
            # Split full HTTP request into individual lines
            headers = message.split("\r\n")
            content_length = 0
            # Extract Content-Length to know how many bytes to expect in the body
            for h in headers:
                if h.lower().startswith("content-length:"):
                    content_length = int(h.split(":")[1].strip())
            # The HTTP body starts after a blank line (\r\n\r\n)
            body = message.split("\r\n\r\n")[1]
            # If body is not fully received (e.g., split into multiple packets), continue
            while len(body) < content_length:
                body += connectionSocket.recv(1024).decode()

            # Parse the form data: "username=user&password=pass" → {'username': ..., 'password': ...}
            params = dict(param.split("=") for param in body.split("&"))
            # Extract the submitted username and password from the parsed data
            username = params.get("username", "")
            password = params.get("password", "")

            # Read the current login statistics from the file
            with open(stats_file, "r") as f:
                lines = f.readlines()
                # Parse the first line to get current number of successful logins
                success = int(lines[0].split("=")[1])
                # Parse the second line to get current number of failed logins
                fail = int(lines[1].split("=")[1])

            # Check submitted credentials against expected values
            if username == "user" and password == "password":
                # Increment success counter
                success += 1
                # Overwrite the stats file with updated values
                with open(stats_file, "w") as f:
                    f.write(f"success={success}\nfail={fail}\n")
                # Prepare the HTML response for a successful login (to be filled below)

```

```

html_response = f"""
<html>
  <head>
    <style>
      body {{ font-family: Arial; margin: 40px; }}
      .success {{ color: green; font-size: 20px; }}
      .fail {{ color: red; font-size: 20px; }}
    </style>
  </head>
  <body>
    <p class="success">Login Success</p>
    <p class="success">Successful logins: {success}</p>
    <p class="fail">Failed logins: {fail}</p>
  </body>
</html>
"""
else:
    fail += 1
    with open(stats_file, "w") as f:
        f.write(f"success={success}\nfail={fail}\n")
    html_response = """
    <html>
      <head>
        <style>
          body { font-family: Arial; margin: 40px; }
          .fail { color: red; font-size: 20px; }
        </style>
      </head>
      <body>
        <p class="fail">Login Failed</p>
      </body>
    </html>
    """

    # Send HTTP 200 OK response header to the client
    connectionSocket.send("HTTP/1.1 200 OK\r\n\r\n".encode())
    # Send the HTML content of the response (either login success or failure)
    connectionSocket.send(html_response.encode())
    # Close the client connection after sending the response
    connectionSocket.close()
    # Exit the current request handler (if inside a thread or function)
    return

# Extract the requested path from the first line of the HTTP request
# Example: "GET /login HTTP/1.1" → "/login"
requested_path = message.split()[1]

```

```

# If the request is for the login page (GET /login), return the login form
if requested_path.startswith("/login"):
    login_form = """
    <html>
        <head>
            <title>Login Page</title>
            <style>
                body {
                    font-family: Arial, sans-serif;
                    margin: 40px;
                }
                label, input {
                    font-size: 16px;
                }
                .submit-btn {
                    font-size: 16px;
                    margin-top: 10px;
                    background-color: #4CAF50;
                    color: white;
                    border: none;
                    padding: 8px 16px;
                    cursor: pointer;
                }
            </style>
        </head>
        <body>
            <h1>Login Page</h1>
            <form action="/login" method="post">
                <label for="username">Please enter your username:</label><br>
                <input type="text" name="username"><br><br>
                <label for="password">Please enter your password:</label><br>
                <input type="password" name="password"><br><br>
                <input class="submit-btn" type="submit" value="Submit">
            </form>
        </body>
    </html>
    """

    connectionSocket.send("HTTP/1.1 200 OK\r\n\r\n".encode())
    connectionSocket.send(login_form.encode())
    connectionSocket.close()
    return

```

```

# If the request path starts with "/" and includes a color query parameter
# Example: GET /?color=red → we need to respond with a color message

```

```

if requested_path.startswith("/") and "color=" in requested_path:
    # Extract the query string part after the "?" in the URL
    # Example: "/?color=red" → "color=red"
    query = requested_path.split("?")[1]
    # Split the query string into individual parameters
    # (in case more than one query parameter is passed)
    params = query.split("&")
    # Initialize a variable to hold the selected color
    color_value = None
    # Look for the "color=" parameter and extract its value
    for param in params:
        if param.startswith("color="):
            color_value = param.split("=")[1]
            break
    # If the color is valid ("red" or "green"), prepare an HTML response
    if color_value == "red" or color_value == "green":
        html_response = f"""
<html>
  <head>
    <title>Color Response</title>
    <style>
      body {{
        font-family: Arial, sans-serif;
        margin: 40px;
      }}
      .result-text {{
        font-size: 16px;
        font-weight: normal;
      }}
      .color-word {{
        color: {color_value};
        font-weight: normal;
      }}
    </style>
  </head>
  <body>
    <p class="result-text">Your color is <span class="color-word">{color_value}</span>
  </body>
</html>
"""
        connectionSocket.send("HTTP/1.1 200 OK\r\n\r\n".encode())
        connectionSocket.send(html_response.encode())
        connectionSocket.close()
    return

```

```

if requested_path == "/":
    html_content = """
    <html>
        <head>
            <title>EE 129 Project 2</title>
            <style>
                body {
                    font-family: Arial, sans-serif;
                    margin: 40px;
                }
                h1 {
                    font-size: 36px;
                    font-weight: bold;
                }
                label, select, input {
                    font-size: 16px;
                }
                .login-btn {
                    font-size: 16px;
                    background-color: #2196F3;
                    color: white;
                    border: none;
                    padding: 8px 16px;
                    cursor: pointer;
                    margin-top: 20px;
                }
            </style>
        </head>
        <body>
            <h1>EE 129 Project 2 Color Picker!</h1>
            <form action="/" method="get">
                <label for="color">Please choose your color:</label>
                <select name="color" id="color">
                    <option value="green" selected>Green</option>
                    <option value="red">Red</option>
                </select>
                <input type="submit" value="Submit">
            </form>
            <form action="/login" method="get">
                <input class="login-btn" type="submit" value="Login">
            </form>
        </body>
    </html>
    """
    connectionSocket.send("HTTP/1.1 200 OK\r\n\r\n".encode())

```

```

        connectionSocket.send(html_content.encode())
        connectionSocket.close()
        return

    connectionSocket.send("HTTP/1.1 404 Not Found\r\n\r\n".encode())
    connectionSocket.send("<html><body><h1>404 Not Found</h1></body></html>".encode())
    connectionSocket.close()

except Exception as e:
    connectionSocket.send("HTTP/1.1 500 Internal Server Error\r\n\r\n".encode())
    connectionSocket.send(f"<html><body><h1>Error: {e}</h1></body></html>".encode())
    connectionSocket.close()

# Main server loop: handles incoming client connections and user commands from terminal
try:
    inputs = [serverSocket, sys.stdin] # Listen to both socket and stdin

    while True:
        print("The server is recieving at http://127.0.0.1:5500/")
        # Use select to wait for input from either:
        # - the server socket (new client connection)
        # - standard input (commands like "exit", "help")
        read_ready, _, _ = select.select(inputs, [], [])

        for source in read_ready:
            # If the server socket is ready, it means a client is trying to connect
            if source == serverSocket:
                # Accept the incoming client connection
                connectionSocket, addr = serverSocket.accept()
                # Start a new thread to handle this client independently
                # Set daemon=True so the server can exit even if client threads are running
                client_thread = threading.Thread(target=handle_client, args=(connectionSocket, addr))
                client_thread.start()
            # If input is ready on stdin, read and handle the command
            elif source == sys.stdin:
                user_input = sys.stdin.readline().strip()
                # If user types "exit", shut down the server and terminate the program
                if user_input == "exit":
                    print("Shutting down server...")
                    serverSocket.close()
                    sys.exit()
                # If user types "help", display available commands
                elif user_input == "help":
                    print("Available commands:")
                    print("  help - show this message")

```

```
        print("  exit - shut down the server")
    # Handle unknown commands
    else:
        print(f"{user_input}: Command Not Found")

# Handle Ctrl+C interruption (e.g., user presses Ctrl+C in terminal)
except KeyboardInterrupt:
    # Print a clean exit message
    print("\nServer interrupted. Exiting...")
    # Close the main server socket to free the port
    serverSocket.close()
    # Exit the program gracefully
    sys.exit()
```




Thank You!

EE-129 Project 2 Report
Tufts University
Department of Electrical and Computer Engineering